*BERN01 VT-2026*

*Group 9*

LUNDS UNIVERSITET

# Monte Carlo Simulation of the q-state Potts model

*Author: Roland Hansson*

*Date: 25/9 2026*


## Abstract

We coded a representation of a Potts Model, and three different Monte Carlo algorithms for updating it. We then investigated their properties, to better compare them and determine their utility.

The work was done in Python. Each member of the group cooperated on general principles, data, graphs, and presentation, but largely wrote their own separate code and lab report.

LLMs were used in an advisory role (“Explain heat-bath again”) and in a recovery role (“where is the bug in this code?”), but overall code design was human.

# Introduction

The Potts model models a system as a lattice of nodes with four neighbours.
The nodes can each have one of *q* states.
The energy of the lattice depends simply on how many of the neighbour-to-neighbour connections that have equal states on each side. (In this model, any pair of different numbers are equally bad.)
This particular model assumes the surface shape is that of a torus; nodes on the edge are neighbours with the other edge.

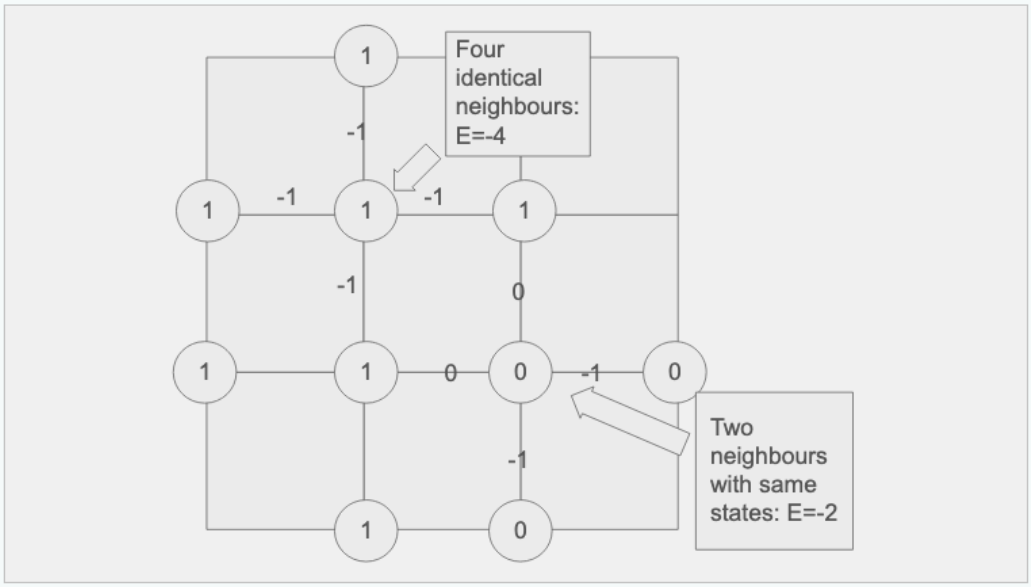



**Fig 1: Representation of a Potts lattice**

The system evolves over time, seeking a lower energy state.
Ideally, such a model would update each node every tick, taking into account the energies involved. However, this would take too much computer power, so instead algorithms are used to randomly effect changes in the model - but according to equations forcing such changes to follow roughly the same patterns observed in nature.

These algorithms are called Monte Carlo algorithms, after a famous casino, since you are essentially throwing dice.

When the algorithm has made enough random changes that the entire lattice could theoretically be covered, this is referred to as a “sweep”.

# Theory

## Monte Carlo Method 1: Metropolis Algorithm

The Metropolis splits the randomized change into two discrete steps:
First propose that a random grid node changes to another colour.
Second, determine whether the proposed change is accepted.
This is accepted automatically when the change is to a lower energy state, and when it's not, the acceptance rate (Eq.1) is dependent on the energy delta and the "environment temperature", T.

*p = e^(-delta_E/T)* *Eq. 1*

One can intuitively see that as T grows, acceptance probability grows closer to e^0 = 1, while a bigger delta_E (a change to a higher energy state) makes the probability shrink again.

## Monte Carlo Method 2: Heat-bath Algorithm

The Heat-bath or Gibbs algorithm picks a position on the lattice, then inserts the value s into that node, where s is randomly drawn from a distribution of values based on the states of the neighbours and the temperature T (Eq. 2).
Notably, this algorithm might simply assign the previous value again, changing nothing.

*p(s=i) = normalize(e^[nr of neighbours=i]/T)* *Eq. 2*

Heat-bath and Metropolis are calculated to give the same result, statistically.

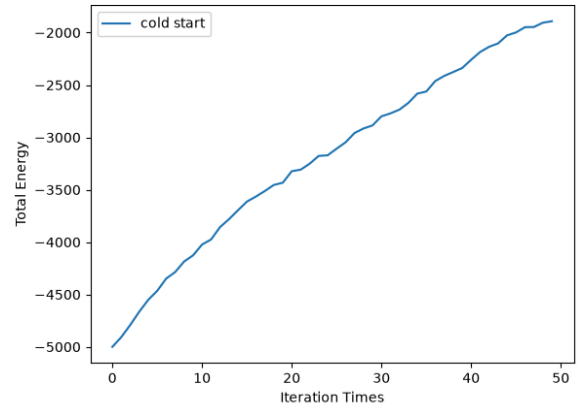
## Monte Carlo Method 3: Wang-Landau Algorithm

Wang-Landau, rather than simulating the state of the model, random-walks possible states of the model in order to build an approximate density plot of it, which can then be saved and used for calculations.

# Results

## Monte Carlo Method 1: Metropolis Algorithm

The Potts model has some interesting behaviour, especially when comparing a hot start (randomized nodes) and a cold start (nodes are all one-colour).



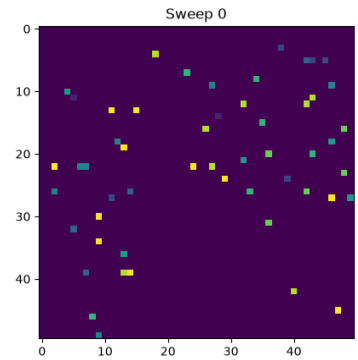
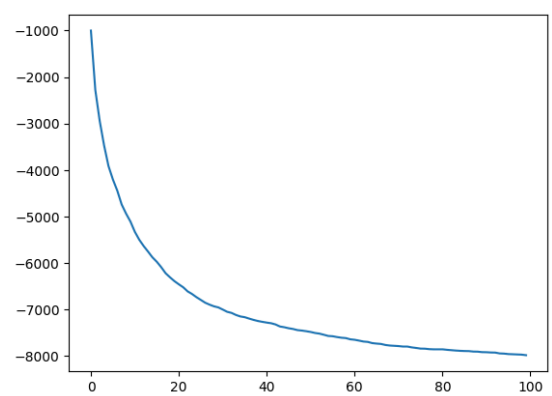




**Fig 2a and 2b: Cold start in hot enviroment (T=0.95)**
The lattice(q=10) starts out in one colour, but becomes more chaotic over time.
The energy increases in basically linear fashion at first, because each sweep, a small group of nodes are added at random.

In Fig 2, we can see that a cold-start lattice in a hot environment gradually turns more chaotic, forced by the environment into a higher-energy state. The final state is not fully chaotic, but consists of small blobs of colour; the equilibrium state where order is created at the same rate as it is destroyed. Meanwhile in Fig 3, the reverse setting does not result in a reverse development.



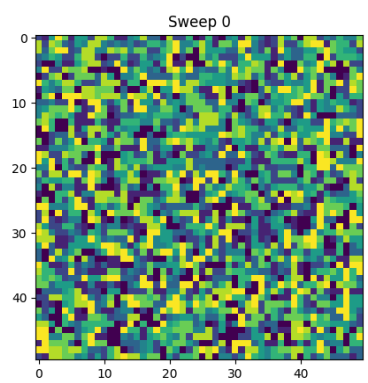




**Fig. 3a and 3b: Hot start in cold environment**
The lattice (q=10) starts out as a chaotic mix of colours, but over time these congeal into slowly growing blobs of colour. It becomes progressively harder to reach a low-energy state, because it takes increasing amounts of random fluctuations for one q-state to overwrite another.

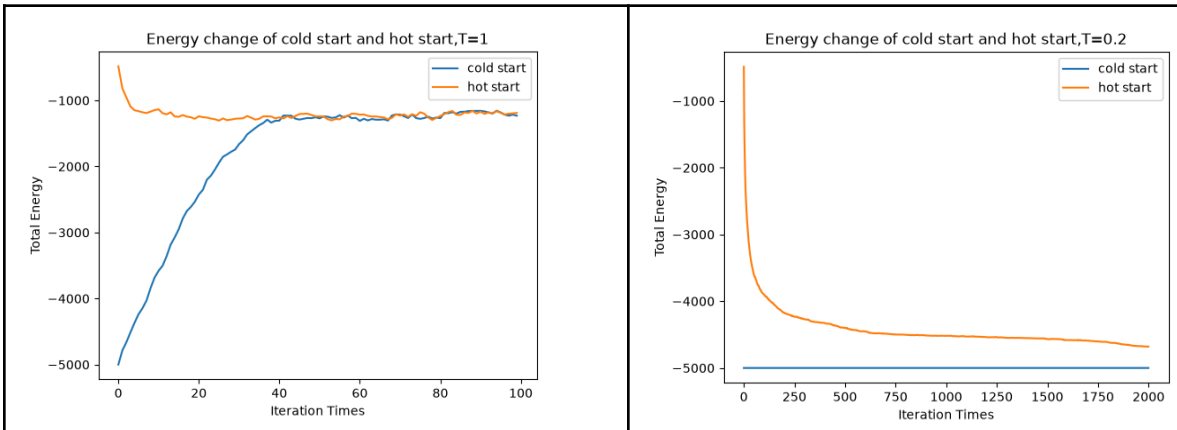



**Fig. 4a and 4b: Energy changes for hot and cold start in hot and cold environments**
Fig 4a(left): The cold-start (fig 2a) and hot-start, when immersed in a hot environment end up in roughly the same chaotic equilibrium, but it is a much shorter journey for the hot-start.
Fig 4b(right): When immersed in a cold environment, the cold-start starts out ordered and remains so, while the hot-start (fig 3a) retain multiple states and only asymptotically approaches the minimum energy state.

The hot hot-start and cold cold-start cases is less interesting, but it's notable that a fully random state is more energetic than the equilibrium (Fig 4a), and that a hot start will not fully freeze into a cold setting in a reasonable time-frame (Fig 4b).

The number of possible states will affect both the equilibrium energy and the environmental temperature where the equilibrium swaps from a ordered to a chaotic state (Fig. 5)

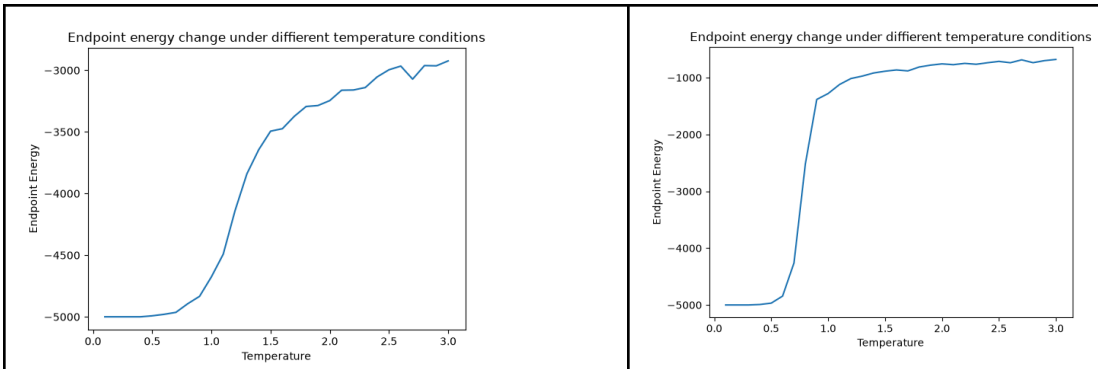



**Fig. 5a and 5b: Equilibrium energy at different temperatures**
5a(left) has q=2, while 5b(right has q=10.

## Metropolis vs. Heat-bath

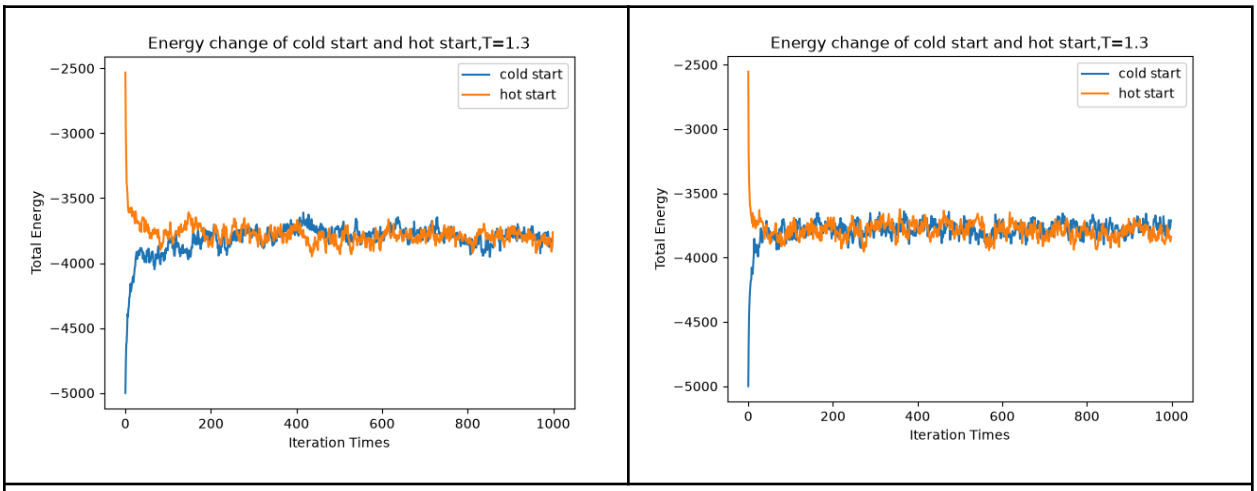



**Fig. 6a and 6b: Two example runs of Metropolis (left) and Heat-bath (right)**

The Heat-bath algorithm gives basically the same values. However, it seems to reach equilibrium slightly more often. (p=0.0004 at confidence level 95%.)

Mean_metro = 62.07, Var_metro = 302

Mean_heat = 40.53, Var_heat = 95

t = mean1 - mean2 - delta / √( var_metro + var_heat / n)

= (62.07 - 40.53 - 0)/ √( ( 302 + 95)/15)

= (21.54 / 5.14)  = 4.19

Looking at t-table, at df=14, it clears 0.0005.

**Table 1: Sweeps before reaching equilibrium**

| # | metropolis sweeps | heatbath sweeps |
|---|---|---|
| 1 | 83 | 43 |
| 2 | 43 | 45 |
| 3 | 90 | 22 |
| 4 | 63 | 42 |
| 5 | 70 | 42 |
| 6 | 39 | 58 |
| 7 | 41 | 43 |
| 8 | 48 | 42 |
| 9 | 70 | 58 |
| 10 | 40 | 36 |
| 11 | 82 | 31 |
| 12 | 51 | 26 |
| 13 | 62 | 43 |
| 14 | 71 | 38 |
| 15 | 78 | 39 |

## Monte Carlo Method 3: Wang-Landau Algorithm

This took hours to run, but produced very smooth data; in Fig 7 we can see a much smoother version of Fig 5. We can note that the transition is much more abrupt in the case of q=10.
This also lets us say with greater precision that the critical temperature - the temperature where the curve swings from one equilibrium to another - for q=10 is 0.7 and for q=2 it's around 1.1.

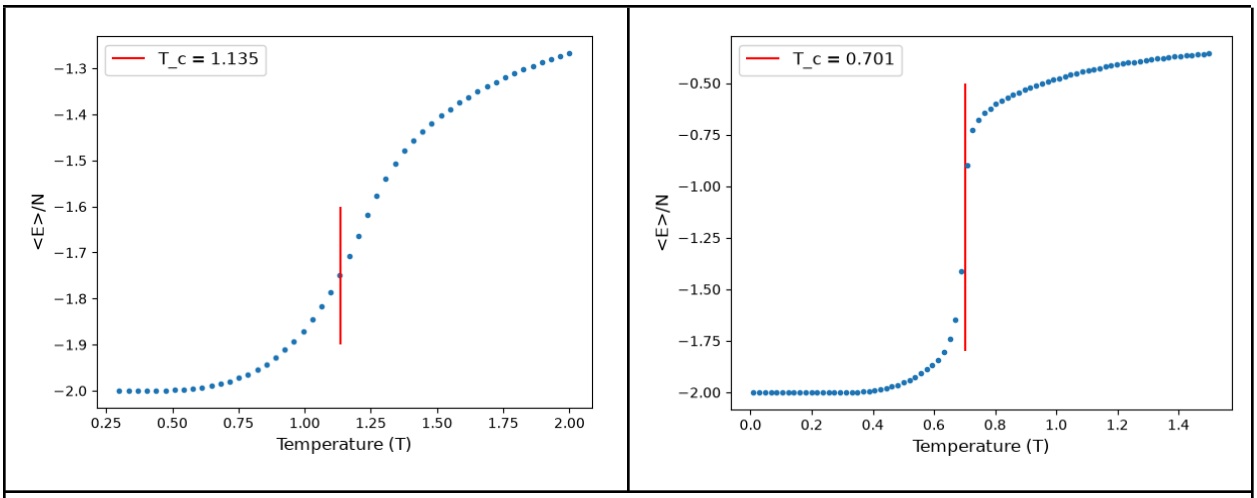



**Fig 7: Calculated Equilibrium energy at different temperatures**
7a(left) has q=2, while 7b(right has q=10.

## Discussion

The heat-bath algorithm seems a bit faster, but it should be noted that this was on a specific lattice size with a specific programmer in a specific language. It's still not great enough of an advantage to say for sure that one will always be better.

Wang-Landau takes a long time to finish due to certain rare states getting few visits. This could be inefficiencies in the code, or possibly because of using Python.

Python is really not the ideal language for a calculation-heavy simulation task, anyway; it might be worth it to put the calculation part into another language, or at least to rely heavily on library functions like matrix calculations. Parallelization could also be useful.

In this model, there are no true sides or corners, and thus no nodes are 'special cases'. However, real-world objects very much have edges and corners, and it could be interesting to explore the consequences; likely those nodes would change more rapidly and influence the rest of the lattice in interesting ways.

## Conclusion

Monte Carlo Potts is an interesting way of modelling certain grid systems.
The heat-bath and Metropolis are both good ways of updating them.
But Python is a less than optimal method for the Wang-Landau algorithm.

## Appendix

Please find the code for this project on:
https://github.com/Ha5248qi/BERN01_group9/tree/Roland-main